



IIT Kharagpur



IIT Madras



IIT Goa



IIT PALAKKAD

IIT Palakkad

Applied Accelerated Artificial Intelligence

Fundamentals of Accelerating Deployments



Satyadhyan Chickerur, PhD

Professor

School of Computer Science and Engineering

KLE Technological University

NVIDIA DLI Ambassador/Instructor



National
Supercomputing
Mission

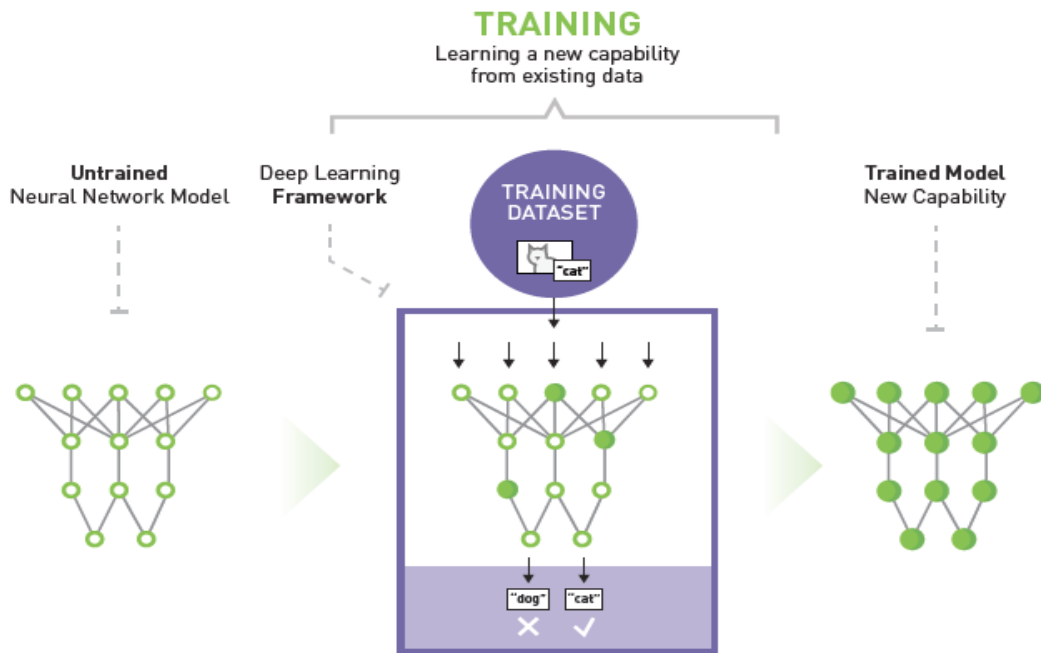


Centre for
Development of
Advanced Computing



Training, Inference and deployment

High-level deep learning workflow showing training, then followed by inference.





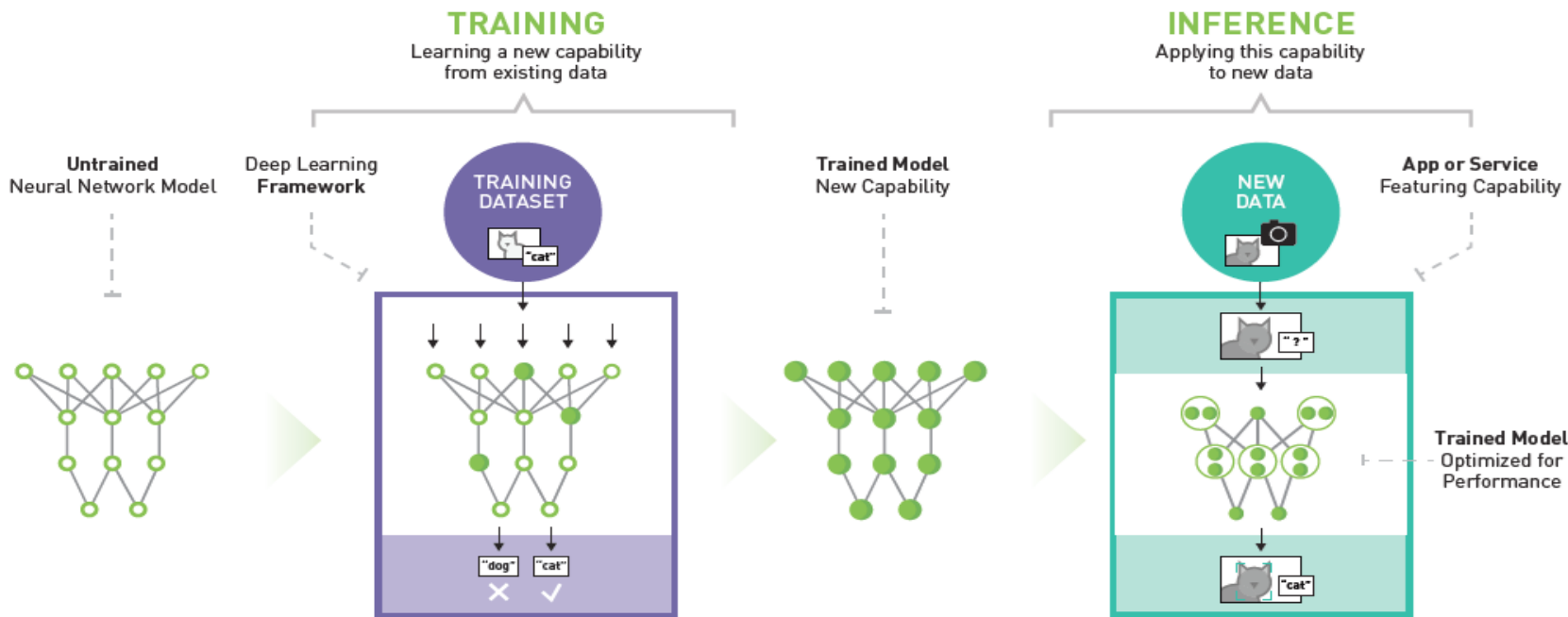
Inference and Deployment

- Training is only half the story
- After a model is trained, we still need to actually use it to make predictions and/or decisions. This is usually called inference.
- Deployed ML inference system – Where do we deploy (Cloud or Edge devices)
- how do we measure the performance of a deployed ML inference system?



Training and Inference.

High-level deep learning workflow showing training, then followed by inference.





Performance of the Deployed System

- Accuracy. How accurate is the model on the queries coming in?
- Latency. How long do we need to wait between when we make a query and when we get a prediction from the system?
- Throughput. If we have a large batch of queries we want a prediction for, how many predictions can we get per second out of the system?
- Energy/power. How much energy is consumed to make one prediction? How much power is used by the deployed ML system in general?
- Model size. How many bytes are needed to store and/or transmit the learned model?
- Memory use. How much memory is used by the deployed ML system?
- Cost. How much money does my deployment cost?

When deploying an ML system, there are trade-offs among ALL of these metrics!

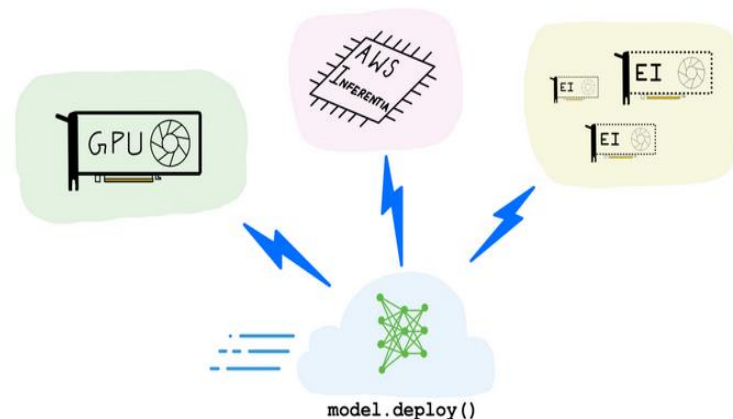


Model Deployment

- Integration of the model into an existing production environment which can take in an input and return an output that can be used in making practical business decisions.

Why it is important ?

- Effective deployment is needed to provide proper prediction to other software systems.
- To maximize the value of the Deep Learning model by reliably extracting the predictions.





How to make a model smaller?

- Model compression for Deep Neural Networks.
 - Goal is to find a smaller, easier-to-compute network with the same or similar accuracy.
- 1. Low-precision inference : Just use low-precision arithmetic for inference.
- 2. Pruning : Idea is to remove weights that are close to zero and/or remove activations that are usually zero. This effectively creates a smaller model. Pruning is often done together with fine-tuning.
- 3. Old-school compression : This is a simple technique of just applying a traditional lossless compression technique, such as Huffman coding, to the learned weights of a network. Of course decompression also should happen at the other end
- 4. Knowledge distillation : The idea is to take a large and/or complex model (called the teacher) and train a smaller and/or simpler network (called the student) to match its output. (distilling ensemble models)
- 5. Efficient architectures : Some neural network architectures are just designed to be efficient at inference time. Examples: MobileNet, ShuffleNet, CirCNN.



Inferencing @ Low Precision

- We will start with a premise that the weights and activations which were in FP32 can be represented using INT8 without incurring significant loss in accuracy.
- What are the advantages : significantly reduced bandwidth and storage. Integer computer faster than FP compute. Also saves area and energy.

Precision	Dynamic Range
FP32	$-3.4 \times 10^{38} \sim +3.4 \times 10^{38}$
FP16	-65504 ~ +65504
INT8	-128 ~ +127

INT8 Operations : ADD – 30x energy saving and 116x area vs. FP32
MUL – 18.5x energy and 27x area vs. FP32

- ← Training precision
- ← No calibration required
- ← Requires calibration

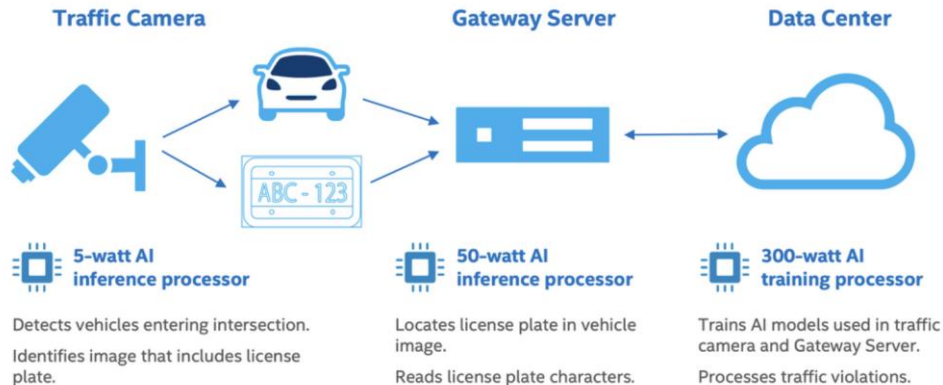
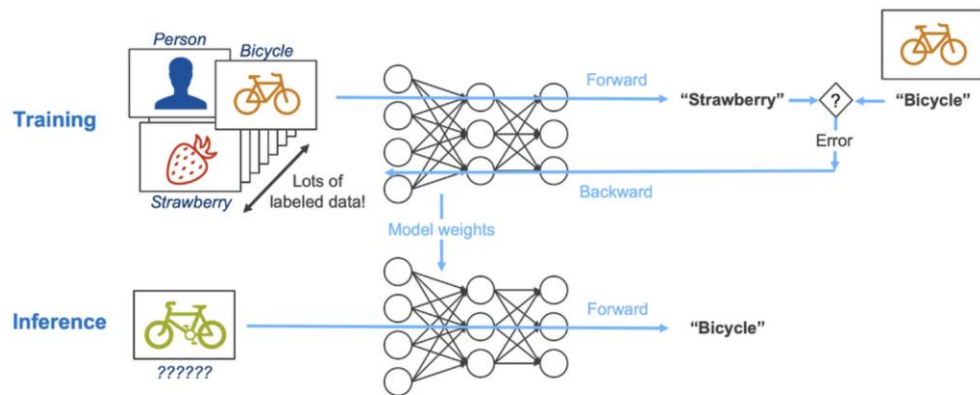
	FP32 Top 1	INT8 Top 1	Difference
Googlenet	68.87%	68.49%	0.38%
VGG	68.56%	68.45%	0.11%
Resnet-50	73.11%	72.54%	0.57%
Resnet-152	75.18%	74.56%	0.61%

Precision calibration for INT8 inference:

- Minimizes information loss between FP32 and INT8 inference on a calibration dataset
- Completely automatic

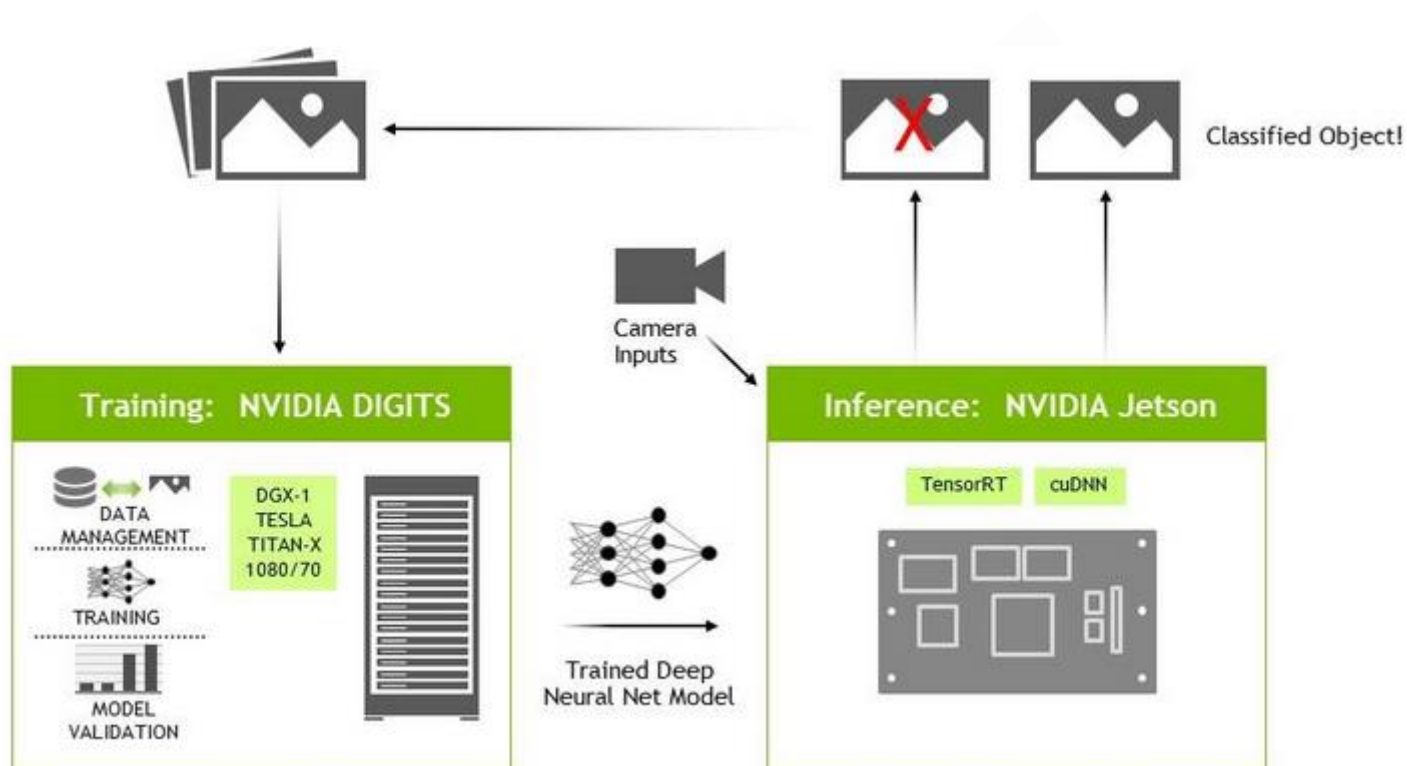


Summary





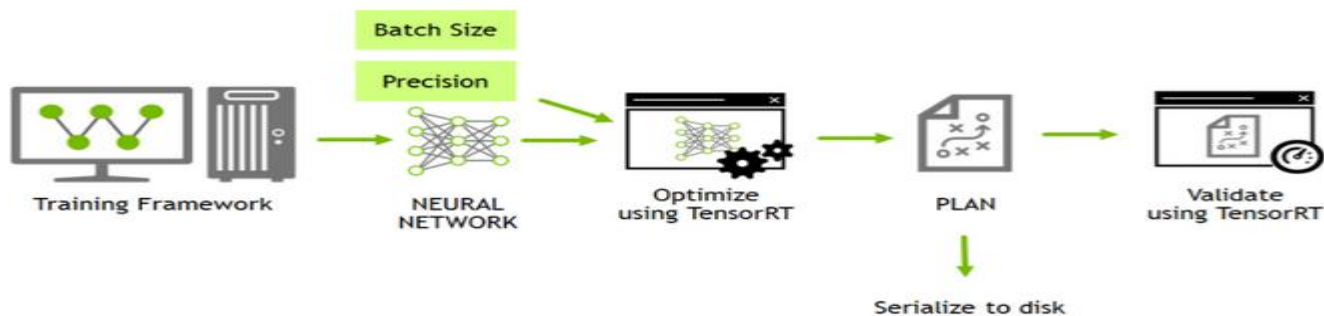
Example Implementation





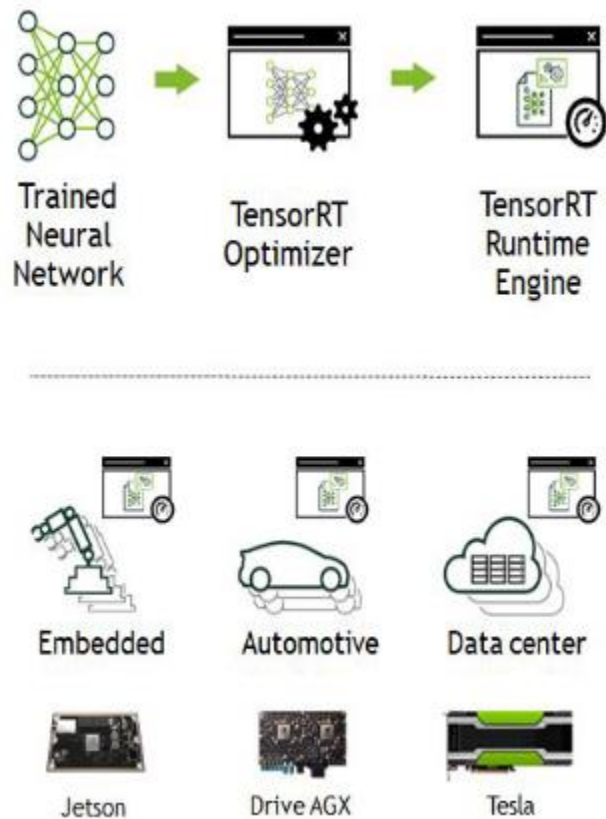
TensorRT- Introduction

- NVIDIA TensorRT™ is a high performance deep learning inference optimizer and runtime that delivers low latency, high-throughput inference for deep learning applications.
- NVIDIA released TensorRT with the goal of accelerating deep learning inference for production deployment.



Why TensorRT?

- A sw platform for high-performance deep learning inference.
- Process images faster! Often need to trade off between speed and accuracy.
- TensorRT-based applications perform up to 40x faster than CPU-only platforms during inference.
- Deploy to hyperscale data centers, embedded, or automotive product platforms.
- Speed up recommender, speech, video and translation in production.
- Take advantage of NVIDIA suite of tools, including DeepStream, NVIDIA Inference Engine.
- Edge processing with NVIDIA Xavier, NVIDIA Jetson TX1, Jetson TX2, Jetson NANO.





TensorRT capabilities

This provides you the overview of what we can do with TensorRT.

- C++ and python APIs
- Programming model {Build phase & Runtime phase}
- Plugins
- Types and precisions
- Quantization
- Tensors and Data formats
- Dynamic shapes
- Deep Learning Accelerators (DLA)
- Updating weights
- Polygraph



Features of TensorRT

1. Reduced Precision/Precision Calibration

- Maximizes throughput with FP16 or INT8 by quantizing models while preserving accuracy.

During the process of training, parameters and activations are in FP32 (Floating Point 32) precision. So to convert them in FP16 or INT8 precision. This optimization not only reduces latency but also gives a significant reduction in model size because FP32 precision gets converted into FP16 or INT8.

	Dynamic Range	Minimum Positive Value
FP32	$-3.4 \times 10^{38} \sim +3.4 \times 10^{38}$	1.4×10^{-45}
FP16	65504 ~ +65504	5.96×10^{-8}
INT8	-128 ~ +127	1



Features of TensorRT

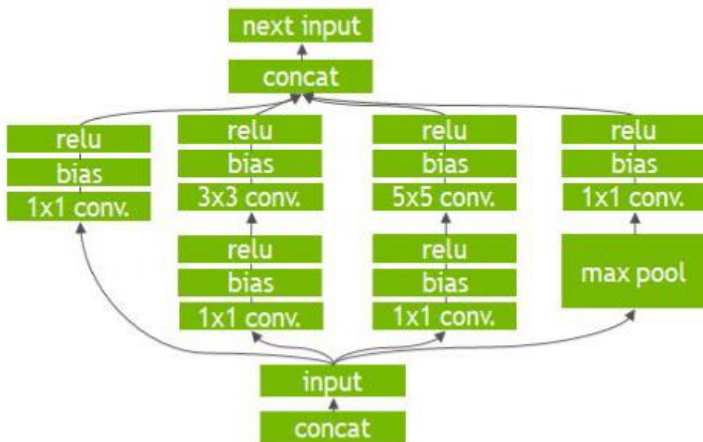
2. Layer and Tensor Fusion :

- Optimizes use of GPU memory and bandwidth by fusing nodes in a kernel.
- TensorRT performs several important transformations and optimizations to the neural network graph. First, layers with unused output are eliminated to avoid unnecessary computation. Next, where possible convolution, bias, and ReLU layers are fused to form a single layer. Figure shows the result of this vertical layer fusion on the original network from (fused layers are labeled CBR). Layer fusion improves the efficiency of running Tensor RT-optimized networks on the GPU.

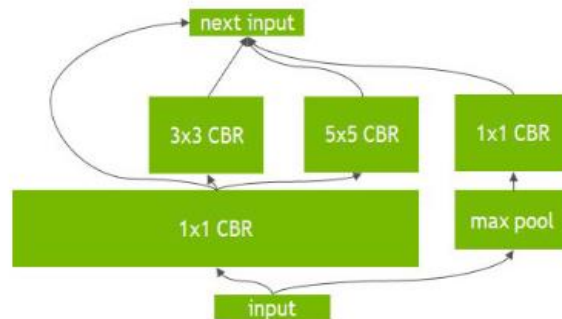
Features of TensorRT

2. Layer and Tensor Fusion :

Unoptimized Network



TensorRT Optimized Network



e.g

Networks	Number of layers (Before)	Number of layers (After)
VGG19	43	27
Inception v3	309	113
ResNet-152	670	159



Features of TensorRT

3. Auto-Tuning Kernel :

- Selects best data layers and algorithms based on the target GPU platform.
- TensorRT will pick the implementation from a library of kernels that delivers the best performance for the target GPU, input data size, filter size, tensor layout, batch size and other parameters.



Features of TensorRT

4. Dynamic Tensor Memory :

- TensorRT reduces memory footprint and improves memory reuse by allocating memory for each tensor only for the duration of its usage, avoiding memory allocation overhead for fast and efficient execution.

5. Multi-Stream Execution :

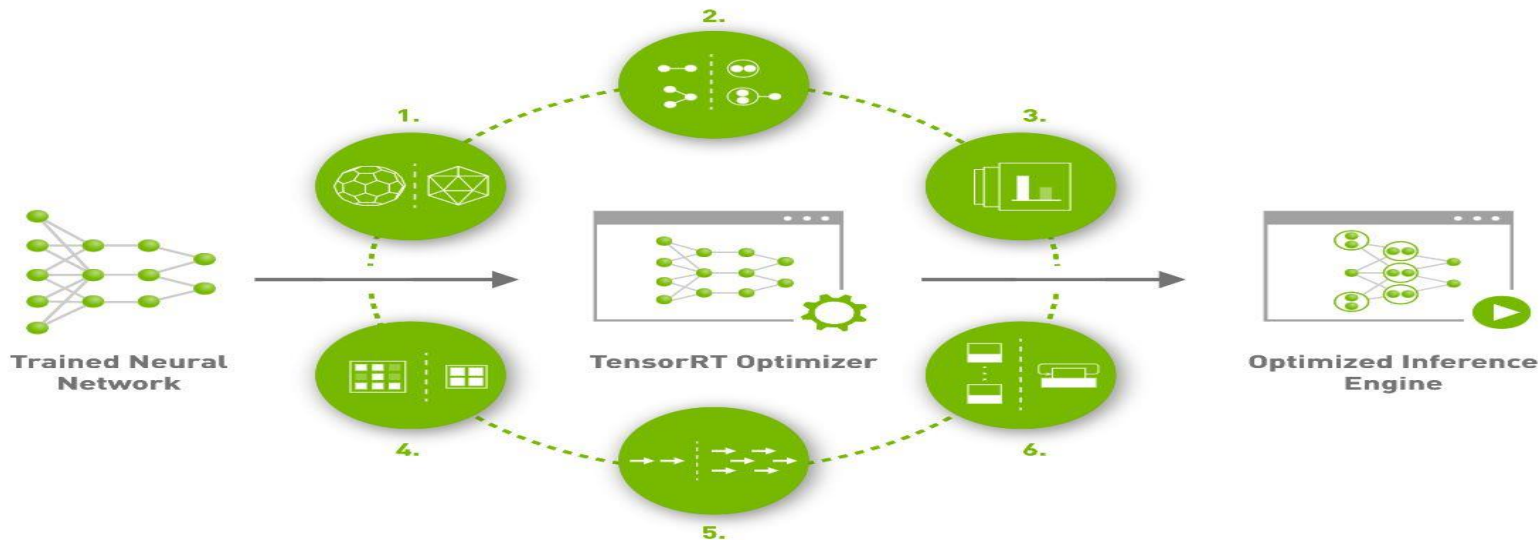
- Uses a scalable design to process multiple input streams in parallel.

6. Time Fusion :

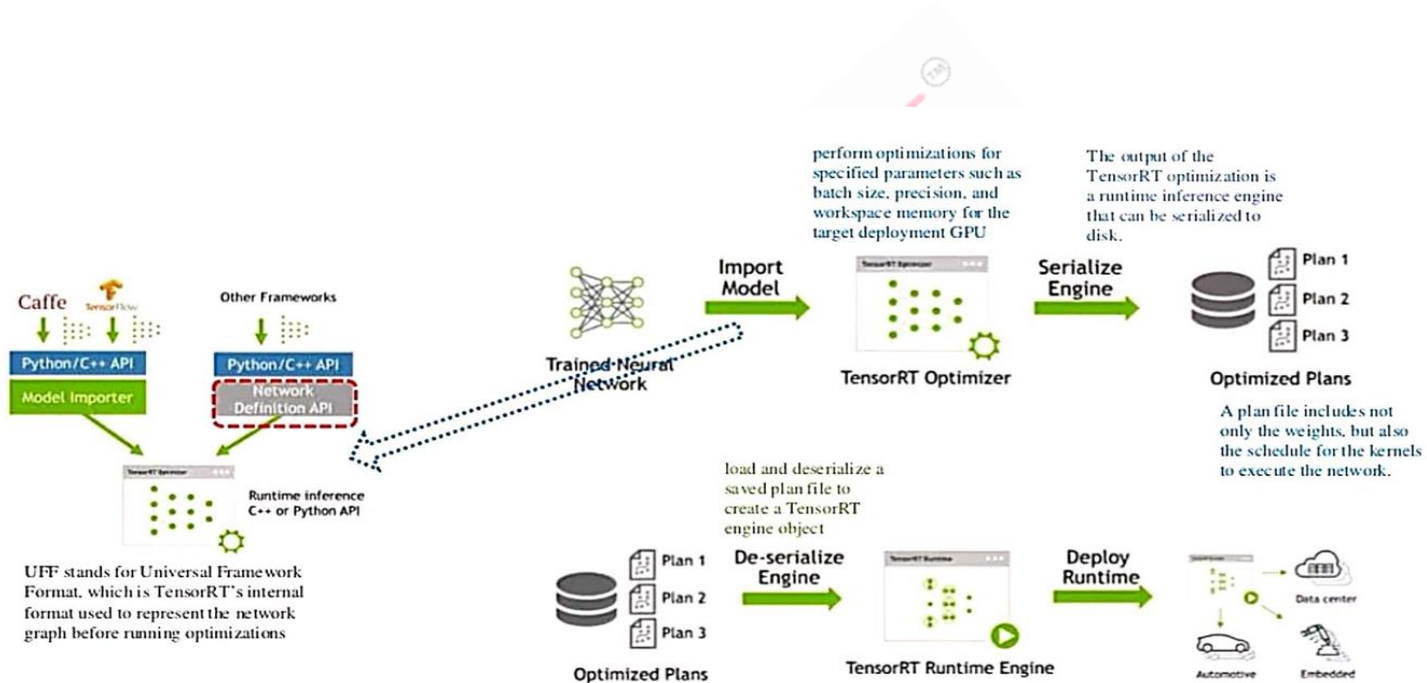
- Optimizes recurrent neural networks over time steps with dynamically generated kernels

Features of TensorRT

TensorRT performs these optimizations automatically under the hood for you.



Deploying a model with TensorRT





TensorRT

TensorRT supports the following layer types.

- Convolution: 2D
- Activation: ReLU, tanh and **sigmoid**
- **Pooling**: max and average
- ElementWise: sum, product or max of two tensors
- LRN: cross-channel only
- Fully-connected: with or without bias
- SoftMax: cross-channel only
- Deconvolution

Thank You



NPTEL

